Unified Security Event Pipeline for Enterprise Threat Detection

CS 4265 — Big Data Analytics: Milestone 3

Olaoluwa Adedamola Omodemi
Kennesaw State University
College of Computing and Software Engineering
CS 4265 — Prof. Christopher Reagan

April 2026

1 Implementation Summary

This milestone delivers a fully operational end-to-end distributed security data pipeline. The system ingests the **LANL Unified Host and Network Dataset** (∼700M events: auth, DNS, flows, process logs) alongside real-time threat intelligence from **URLHaus**, processes all sources through a Bronze → Silver → Gold medallion architecture, and writes enriched output to Amazon S3 in Parquet format.

All pipeline stages are implemented and executable. Configuration is fully externalized via `.env`; no credentials or paths are hardcoded.

Repository:

2 Big Data Stack

Layer	Technology	Concept
Storage	Amazon S3	Object store, prefix-per-layer, lifecycle
Processing	Apache Spark	DAG execution, S3A connector
Data Store	Apache HBase	Wide-column, LSM-Trees (M4)
Format	Apache Parquet	Columnar + Snappy compression
Querying	Spark SQL	Partition-aware distributed queries

Data sources:

- LANL auth/DNS/flows/proc logs (CSV/bz2) — https://csr.lanl.gov/data/cyber1/
- URLHaus threat feed (JSON API) — https://urlhaus.abuse.ch/api/

3 Data Flow Description

Bronze (Ingestion): Raw LANL CSV files are loaded into PySpark DataFrames with schema enforced at read time. URLHaus threat intelligence is fetched via the REST API using `scripts/fetch_urlhaus.py`.

Silver (Normalization): Each source is cleaned, typed, and written to Parquet under: `s3a://security-pipeline-lanl/silver/<dataset>/`. Processing scripts include:

- `src/processing/auth_to_parquet.py`
- `src/processing/dns_to_parquet.py`
- `src/processing/flows_to_parquet.py`
- `src/processing/proc_to_parquet.py`
- `scripts/urlhaus_to_parquet.py`

Gold (Enrichment): `src/scripts/enrich.py` joins all five silver datasets, flags flows whose destination matches URLHaus threat intel, buckets events into hourly windows, and aggregates to a user-activity summary.

The output is written to: `s3a://security-pipeline-lanl/gold/user_activity_summary/` partitioned by `time_bucket`.

3.1 Join Strategy

All lookup tables are deduplicated before joining to prevent row fanout:

- **Auth:** deduplicated on `(source_computer, time)` — one user per host per second.
- **Proc:** deduplicated on `(computer, time)`.
- **URLHaus:** hostname extracted via `regexp_extract` from the full URL, then deduplicated on host.

All joins are performed as left outer joins so that flows without a matching auth or process record are preserved. A null `user` is itself a signal of unauthenticated traffic.

4 Data Dictionary

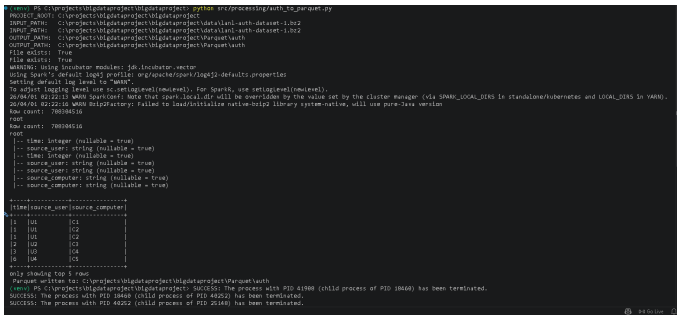

Figure 1: Example Parquet output for auth dataset is also what would was showing for proc,flow,dns,urlhaus.

4.1 Silver Layer Schemas

auth (`silver/auth/`):

Field	Type	Description
`time`	Long	Seconds since epoch
`source_user`	String	Authenticating user
`source_computer`	String	Host initiating auth

proc (`silver/proc/`):

Field	Type	Description
`time`	Long	Seconds since epoch
`user_domain`	String	Domain of process owner
`computer`	String	Host running the process
`process_name`	String	Executable name
`start_end`	String	Process start or end event

flows (`silver/flows/`):

Field	Type	Description
`time`	Long	Seconds since epoch
`duration`	Long	Flow duration (s)
`src_computer`	String	Source host
`src_port`	Integer	Source port
`dst_computer`	String	Destination host
`dst_port`	Integer	Destination port
`protocol`	String	Network protocol
`packets_count`	Long	Packets transmitted
`bytes_count`	Long	Bytes transmitted

dns (`silver/dns/`):

Field	Type	Description
`time`	Long	Seconds since epoch
`SourceComputer`	String	Host issuing DNS query
`ComputerResolved`	String	Resolved hostname

urlhaus (`silver/urlhaus/`):

Field	Type	Description
`url`	String	Full malicious URL
`threat`	String	Threat classification label

4.2 Gold Layer Schema

user_activity_summary (`gold/user_activity_summary/`):

Field	Type	Description
`time_bucket`	Long	Hourly epoch floor (partition key)
`user`	String	Authenticated user (nullable)
`src_computer`	String	Source host
`total_flows`	Long	Flow count in window
`total_bytes`	Long	Byte volume in window
`total_packets`	Long	Packet count in window
`active_processes`	Long	Distinct process names seen
`malicious_hits`	Long	URLHaus host matches

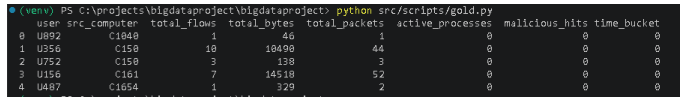

Figure 2: Output of the enrichment layer for user activity

5 Pipeline Documentation

5.1 Stage Configuration Options

Each stage reads its configuration from `.env`. The table below maps the key variables to the scripts they affect:

Variable	Stage	Effect
`AWS_ACCESS_KEY_ID`	All	S3 credentials
`AWS_SECRET_ACCESS_KEY`	All	S3 credentials
`AWS_DEFAULT_REGION`	All	S3 region
`SILVER_PARQUET_URI`	Silver	Silver S3 base path
`GOLD_PARQUET_URI`	Gold	Gold S3 base path
`PARQUET_OUTPUT_ROOT`	Silver	Local fallback path
`DATA_DIR`	Bronze	Raw input directory
`SPARK_MASTER`	All	Spark master URL
`SPARK_DRIVER_MEMORY`	All	Driver heap (default 8g)
`SPARK_JARS_PACKAGES`	All	Maven JAR coordinates
`URLHAUS_API_KEY`	Acquire	URLHaus API auth

When `SILVER_PARQUET_URI` is unset, output defaults to `Parquet/<dataset>/` under the project root, enabling fully local runs without S3.

6 Transformation Decisions

Time bucketing: Raw timestamps are bucketed hourly to avoid excessive small-file partitions while preserving useful temporal granularity.

Aggregation: The gold layer aggregates to `(time_bucket, user, src_computer)` to reduce data size while retaining key behavioral signals.

Malicious flag: Threat matches are precomputed during enrichment, avoiding repeated joins during analysis.

Null handling: Null values from left joins are preserved, as they can indicate suspicious activity (e.g., unauthenticated traffic).

S3 normalization: All `s3://` paths are converted to `s3a://` to ensure correct Spark filesystem usage.

7 Running the Pipeline

7.1 Pipeline Execution Sequence

The pipeline follows a medallion architecture: Bronze (raw ingestion), Silver (normalized Parquet), Gold (enriched analytics-ready data). Here's the recommended sequence to run the full pipeline:

1. Data Preparation

`Ensure raw LANL files`
e.g., `data/lanl-auth-dataset-1.bz2`, `dns.txt.gz`, etc.

`Fetch threat intelligence`
```
python scripts/fetch_urlhaus.py
```

`Optional: Generate samples`
```
python scripts/create_samples.py
```

2. Bronze Layer: Ingestion

`Load auth data`
```
python src/ingestion/ingest_auth_logs.py
```

`Load DNS data`
```
python src/ingestion/ingest_dns.py
```

`Load flows data`
```
python src/ingestion/ingest_flows.py
```

`Load proc data`
```
python src/ingestion/ingest_proc.py
```

3. Silver Layer: Normalization

`Auth to Parquet`
```
python src/processing/auth_to_parquet.py
```

`DNS to Parquet`
```
python src/processing/dns_to_parquet.py
```

`Flows to Parquet`
```
python src/processing/flows_to_parquet.py
```

`Proc to Parquet`
```
python src/processing/proc_to_parquet.py
```

4. Gold Layer: Enrichment

`Correlate with threats`
```
python src/scripts/enrich.py
```

7.2 Evidence of Successful Execution

The following is representative console output from a successful gold-layer run:

Figure 3: The gold layer running
Note:There will be a massive output at the end of it that is just the program trying to clear the tmp file and failing

7.3 Sample Gold Output (`df.show()`)

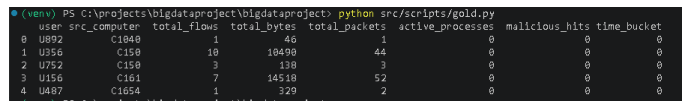

Figure 4: Output of the enrichment layer for user activity

8 Challenges and Solutions

S3A Connector: PySpark does not include the S3A connector by default. Required dependencies (`hadoop-aws` and `aws-java-sdk-bundle`) were added via `spark.jars.packages`.

Version Compatibility: A mismatch between Spark's bundled Hadoop (3.3) and `hadoop-aws:3.4.0` caused runtime errors. This was resolved by pinning the connector to `hadoop-aws:3.3.6`.

Vectored I/O: Parquet vectored reads caused compatibility issues and were disabled through Spark and JVM configuration.

Configuration Parsing: Hadoop duration strings (e.g., `"24h"`) caused parsing errors in the local environment. All duration values were overridden with numeric millisecond values.

Windows Setup: Incorrect `HADOOP_HOME` configuration caused startup failures. This was fixed by normalizing the path to avoid duplicate `bin` directories.

Schema Alignment: Column name mismatches between datasets were resolved by inspecting DataFrame schemas and updating join logic accordingly.

9 Performance Observations

Running locally (`local[*]`, 8 GB driver) against S3A on sample data:

- **Session startup** (incl. JAR resolution): ∼30s first run, ∼8s cached.
- **Silver write per dataset**: 15–60s depending on source size.
- **Gold enrichment** (all five datasets): 3–8 minutes.
- **Primary bottleneck**: S3A round-trip latency on small Parquet parts. Production runs on EMR against the full 700M-row dataset would benefit from S3 Transfer Acceleration, larger part files, and executor parallelism scaled to cluster size.

10 Stack Implementation Status

Layer	M3 Status	Notes
Storage (S3)	**Done**	Bronze/Silver/Gold prefixes
Format (Parquet)	**Done**	Snappy, hourly partitions
Processing (Spark)	**Done**	PySpark DAG, S3A
Querying (SQL)	**Done**	DataFrame API, gold layer
Data Store (HBase)	Planned (M4)	Schema defined

11 Remaining Work for M4

- **HBase writes:** Implement gold-to-HBase indexing with composite row-key design (`user|time_bucket`) to prevent region hotspotting and enable low-latency lookups.
- **Error handling:** Add path-existence checks before reads; wrap the write phase in try/except with structured logging; implement retry logic for S3A transient failures.
- **Graceful partial failure:** Allow the pipeline to continue if a non-critical dataset (e.g. DNS) is absent rather than aborting the entire job.
- **Full dataset validation:** Run against the complete LANL 700M-row dataset on EMR and measure throughput, latency, and data fidelity.
- **Threat hunting notebooks:** Wire Jupyter notebooks against the gold Parquet layer for SQL-based analyst workflows.
- **Schema validation metrics:** Document null rates, cardinality, and row counts per dataset as part of M4 data quality report.

12 Final Technical Baseline

- **Connector:** `hadoop-aws:3.3.6` + `aws-java-sdk-bundle:1.12.367`
- **I/O mode:** Legacy sequential Parquet reader; vectored I/O disabled at JVM, Spark SQL, and S3A layers
- **Config style:** Pure numeric millisecond values; no time-unit suffixes anywhere
- **Windows bootstrap:** `PYSPARK_PYTHON`/`PYSPARK_DRIVER_PYTHON` set to `sys.executable`; `HADOOP_HOME` normalized to strip trailing \`bin` before JVM init
- **Duration override:** `fs.s3a.multipart.purge.age=86400000` (ms) explicitly set to bypass Hadoop's `"24h"` default string that causes `NumberFormatException`